Homework 05

Question 1 Answers

Part a

$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.5942$. Seed: 43251.

```
f_true <- function(X) 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2

# KNN regression with equal weights; returns one column per k.
knn_reg <- function(X_train, y_train, X_test, ks) {
  d2 <- outer(rowSums(X_test^2), rowSums(X_train^2), "+") -
    2 * X_test %*% t(X_train)
  t(apply(d2, 1, function(d) {
    (cumsum(y_train[order(d)]) / seq_along(d))[ks]
  }))
}

set.seed(43251)
n <- 400
ks <- seq(1, 29, 2)
x0 <- matrix(c(0.5, 0.7, 1), nrow = 1)
f0 <- f_true(x0)

X <- matrix(rnorm(n * 3), n, 3)
y <- f_true(X) + rnorm(n, sd = 0.1)
pred_one <- knn_reg(X, y, x0, ks)
colnames(pred_one) <- ks

print(f0)
```

```
[1] 0.5942177
```

```
print(round(pred_one, 4))
```

```
          1      3      5    7      9     11     13    15     17     19     21
[1,] 0.7888 0.6622 0.6643 0.61 0.6319 0.4925 0.5223 0.581 0.5738 0.5751 0.5801
         23     25     27     29
[1,] 0.5519 0.5215 0.4953 0.4973
```

Part b

```
n_sim <- 200
pred <- matrix(NA, n_sim, length(ks), dimnames = list(NULL, ks))

for (s in 1:n_sim) {
  X <- matrix(rnorm(n * 3), n, 3)
  y <- f_true(X) + rnorm(n, sd = 0.1)
  pred[s, ] <- knn_reg(X, y, x0, ks)
}

bias2 <- (colMeans(pred) - f0)^2
variance <- colMeans(sweep(pred, 2, colMeans(pred))^2)
```

```r
mse <- colMeans((pred - f0)^2)

print(round(rbind(bias2, variance, mse), 5))
```

```
              1       3       5       7       9      11      13      15
bias2    0.00000 0.00086 0.00167 0.00208 0.00269 0.00389 0.00460 0.00499
variance 0.03718 0.01983 0.01473 0.01087 0.00947 0.00951 0.00940 0.00854
mse      0.03718 0.02069 0.01640 0.01294 0.01216 0.01340 0.01399 0.01354
             17      19      21      23      25      27      29
bias2    0.00664 0.00799 0.00923 0.01057 0.01163 0.01309 0.01481
variance 0.00821 0.00843 0.00841 0.00810 0.00809 0.00778 0.00747
mse      0.01486 0.01642 0.01764 0.01867 0.01972 0.02087 0.02228
```

```r
print(max(abs(mse - (bias2 + variance))))
```

```
[1] 6.591949e-17
```

```r
plot(
  ks, mse, type = "l", lwd = 2, col = "black",
  ylim = range(0, bias2, variance, mse),
  xlab = "k", ylab = "Value"
)
lines(ks, bias2, lwd = 2, col = "#1F5A94")
lines(ks, variance, lwd = 2, col = "#C84A16")
legend(
  "top", bty = "n", lwd = 2, col = c("black", "#1F5A94", "#C84A16"),
  legend = c("MSE", "Squared bias", "Variance")
)
```

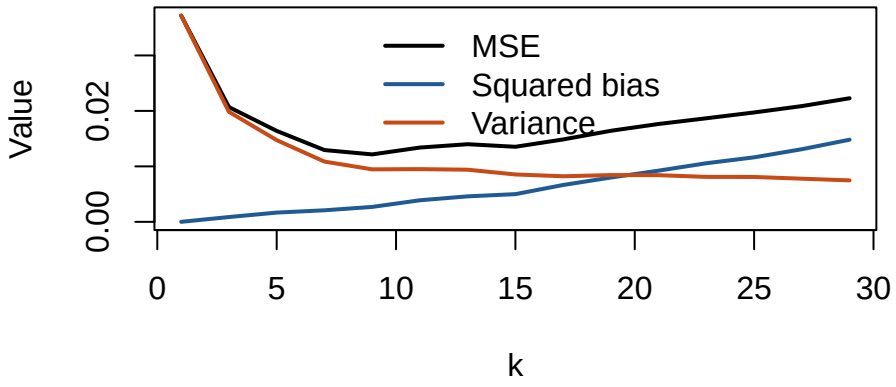


Figure 1: Squared bias, variance, and MSE of the KNN estimate at x0 (200 simulations).

The identity holds to machine precision ($\approx 10^{-17}$). Squared bias starts near 0 at $k = 1$ and grows as the neighborhood reaches points where f differs from $f(\mathbf{x}_0)$; variance falls from 0.037 at $k = 1$ toward σ^2/k as more responses are averaged. MSE is minimized around $k = 9$. This matches ridge and lasso, where increasing λ also trades higher bias for lower variance: small k plays the role of small λ (flexible, high variance) and large k that of large λ (smooth, high bias).

Question 2 Answers

Part a

Seed: 43252.

```
library(glmnet)

# rho = 0 gives independent covariates; rho = 0.8 gives equicorrelated ones.
gen_X <- function(n, p, rho) {
  Z <- matrix(rnorm(n * p), n, p)
  if (rho > 0) Z <- sqrt(rho) * rnorm(n) + sqrt(1 - rho) * Z
  Z
}

# Centers/scales with divisor n from the training portion only; unpenalized intercept is ybar.
lasso_predict <- function(X_train, y_train, X_new, lambda) {
  mu <- colMeans(X_train)
  s <- sqrt(colMeans(sweep(X_train, 2, mu)^2))
  Xs <- sweep(sweep(X_train, 2, mu), 2, s, "/")
  fit <- glmnet(
    Xs, y_train - mean(y_train), lambda = lambda,
    standardize = FALSE, intercept = FALSE, control = list(thresh = 1e-12)
  )
  X_new_s <- sweep(sweep(X_new, 2, mu), 2, s, "/")
  list(pred = mean(y_train) + as.matrix(X_new_s %*% fit$beta), beta = fit$beta)
}

set.seed(43252)
p <- 30
lambda_grid <- 10^seq(0, -4, length.out = 41)
results <- data.frame()

for (rho in c(0, 0.8)) {
  X_train <- gen_X(400, p, rho)
  y_train <- f_true(X_train) + rnorm(400, sd = 0.1)
  X_test <- gen_X(1000, p, rho)
  y_test <- f_true(X_test) + rnorm(1000, sd = 0.1)

  knn_mse <- mean((y_test - knn_reg(X_train, y_train, X_test, 5))^2)

  folds <- sample(rep(1:10, length.out = 400))
  cv_mse <- rowMeans(sapply(1:10, function(k) {
    out <- lasso_predict(
      X_train[folds != k, ], y_train[folds != k],
      X_train[folds == k, , drop = FALSE], lambda_grid
    )
    colMeans((y_train[folds == k] - out$pred)^2)
  }))
  lambda_best <- lambda_grid[which.min(cv_mse)]
  final <- lasso_predict(X_train, y_train, X_test, lambda_best)

  results <- rbind(results, data.frame(
    setting = ifelse(rho == 0, "1 (independent)", "2 (corr 0.8)"),
    knn5_test_mse = knn_mse,
```

```
    lasso_test_mse = mean((y_test - final$pred)^2),
    lasso_lambda = lambda_best,
    nonzero_slopes = sum(final$beta != 0)
  ))
}

print(results, digits = 4)
```

```
          setting knn5_test_mse lasso_test_mse lasso_lambda nonzero_slopes
1 1 (independent)        0.5681         0.2867      0.03981              5
2    2 (corr 0.8)        0.1729         0.2365      0.01995              7
```

Part b

5NN goes from much worse than lasso in Setting 1 (0.568 vs. 0.287) to better in Setting 2 (0.173 vs. 0.236). With independent covariates, squared Euclidean distance is a sum of 30 comparable terms, 27 of which are irrelevant, so neighbors are far apart in (x_1, x_2, x_3). With correlation 0.8, $X_j = \sqrt{0.8}\,W + \sqrt{0.2}\,Z_j$: distances are dominated by the shared factor W, the data are effectively low-dimensional, and neighbors sharing W also have similar x_1, x_2, x_3 (each only $\sqrt{0.2}$ of independent spread). Lasso instead weights covariates individually, setting irrelevant slopes to zero (only 5 and 7 nonzero), so irrelevant covariates barely affect it, while KNN counts every covariate equally in the distance.

Part c

Lasso can only fit a linear function, so it cannot represent $-0.3X_3^2$ (uncorrelated with every linear term in Setting 1) or the curvature of $\sin(X_2)$; that approximation error puts a floor on its test MSE no matter how the penalty is tuned. KNN is not limited to linear forms, so part of the comparison is flexibility vs. dimension rather than correlation alone. The results come from one dataset per setting, one correlation structure, one k, and one response function, so they do not justify a general claim that correlation always helps KNN or that one method is always better — correlation helped here because it lowered the effective dimension in a way aligned with the signal, and with a linear, sparse response lasso would likely win in both settings.

Question 3 Answers

Part a

Seed: `43253` (used for parts a and b).

```
gen_latent <- function(n, A) {
  m <- ncol(A)
  Z <- matrix(rnorm(n * m), n, m)
  X <- Z %*% t(A) + matrix(rnorm(n * 100), n, 100)
  list(X = X, y = Z[, 1] + Z[, 2] + rnorm(n))
}

run_once <- function(m, ks) {
  A <- matrix(runif(100 * m, -2, 2), 100, m)
  train <- gen_latent(200, A)
  test <- gen_latent(200, A)
  colMeans((test$y - knn_reg(train$X, train$y, test$X, ks))^2)
}

set.seed(43253)
k_grid <- seq(2, 82, 8)
```

```r
one_run <- rbind(m3 = run_once(3, k_grid), m30 = run_once(30, k_grid))
colnames(one_run) <- k_grid
print(round(one_run, 3))
```

```
        2    10    18    26    34    42    50    58    66    74    82
m3  1.616 1.151 1.159 1.243 1.283 1.353 1.404 1.447 1.506 1.554 1.594
m30 2.886 2.040 1.938 2.005 2.010 2.080 2.054 2.097 2.123 2.161 2.176
```

Part b

```r
n_rep <- 50
mse_m3 <- t(replicate(n_rep, run_once(3, k_grid)))
mse_m30 <- t(replicate(n_rep, run_once(30, k_grid)))
avg_m3 <- colMeans(mse_m3)
avg_m30 <- colMeans(mse_m30)

print(round(rbind(k = k_grid, m3 = avg_m3, m30 = avg_m30), 3))
```

```
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]   [,7]   [,8]   [,9]  [,10]  [,11]
k   2.000 10.000 18.000 26.000 34.000 42.000 50.000 58.000 66.000 74.000 82.000
m3  1.712  1.296  1.316  1.365  1.419  1.472  1.524  1.580  1.641  1.703  1.768
m30 3.088  2.351  2.337  2.368  2.402  2.438  2.471  2.499  2.535  2.566  2.596
```

```r
plot(
  k_grid, avg_m3, type = "b", pch = 16, lwd = 2, col = "#1F5A94",
  ylim = range(1, avg_m3, avg_m30),
  xlab = "k", ylab = "Average test MSE"
)
lines(k_grid, avg_m30, type = "b", pch = 17, lwd = 2, col = "#C84A16")
abline(h = 1, lty = 3, col = "grey50")
legend(
  "right", bty = "n", lwd = 2, pch = c(16, 17), col = c("#1F5A94", "#C84A16"),
  legend = c("m = 3", "m = 30")
)
```

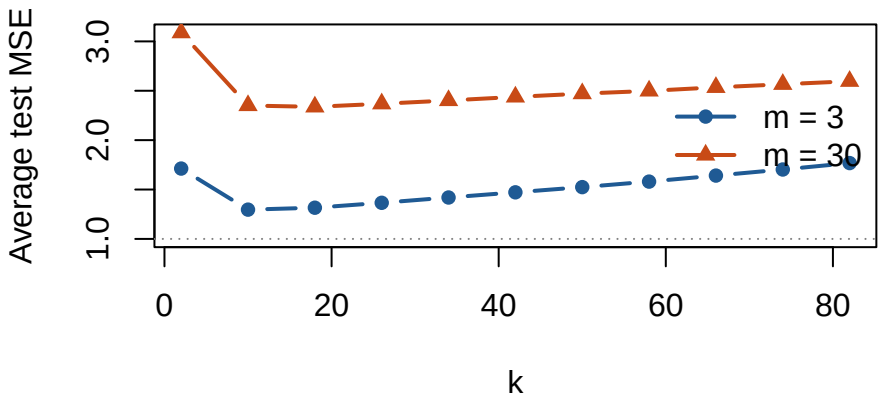


Figure 2: Average KNN test MSE over 50 repetitions.

The dotted line is the irreducible error $\text{Var}(\epsilon) = 1$.

Part c

Without η_i, $\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i$ would lie exactly in the m-dimensional column space of $\mathbf{A}$, and Euclidean distance in $\mathbf{X}$ would be a distance in $\mathbf{Z}$. KNN's difficulty is therefore governed by the latent dimension m, not the 100 observed coordinates. With $m = 3$, 200 points fill a 3-dimensional space well, so neighbors in $\mathbf{X}$ are close in (Z_1, Z_2). With $m = 30$, the distance depends on all 30 latent variables, only two of which drive Y; neighbors are far apart in the 30-dimensional space and are not close in (Z_1, Z_2), so test MSE is much higher (about 2.3 vs. 1.3 at the best k).

The noise η_i adds independent variation in all 100 directions, so $\mathbf{X}_i$ is spread around the m-dimensional subspace rather than on it, and the data have full rank 100. Because each η_{ij} has variance 1 versus about $4m/3$ for each coordinate of $\mathbf{A}\mathbf{Z}_i$, the noise adds a roughly constant amount to all squared distances plus some random perturbation of the neighbor ranking; it blurs neighborhoods but the latent structure still dominates.

Question 4 Answers

Part a

```
zip_train <- as.matrix(read.csv("data/knn/zip-train.csv.gz", header = FALSE))[1:1000, ]
zip_test <- as.matrix(read.csv("data/knn/zip-test.csv.gz", header = FALSE))[1:500, ]
X_train <- zip_train[, -1]
y_train <- zip_train[, 1]
X_test <- zip_test[, -1]
y_test <- zip_test[, 1]

knn5_classify <- function(x0, X_train, y_train,
                          distance = c("euclidean", "manhattan"), k = 5) {
  distance <- match.arg(distance)
  diffs <- t(X_train) - x0
```

```
  d <- if (distance == "euclidean") sqrt(colSums(diffs^2)) else colSums(abs(diffs))
  nn <- order(d, seq_along(d))[1:k]             # distance ties -> smaller row
  votes <- tabulate(y_train[nn] + 1, nbins = 10)
  which.max(votes) - 1                          # vote ties -> smallest digit
}
```

The data are the course CSV copies of `zip.train` and `zip.test`.

Part b

```
pred_euc <- apply(X_test, 1, knn5_classify, X_train, y_train, "euclidean")
pred_man <- apply(X_test, 1, knn5_classify, X_train, y_train, "manhattan")

print(c(error_euclidean = mean(pred_euc != y_test),
        error_manhattan = mean(pred_man != y_test)))
```

```
error_euclidean error_manhattan
          0.122           0.138
```

```
conf_euc <- table(predicted = factor(pred_euc, 0:9), true = factor(y_test, 0:9))
conf_man <- table(predicted = factor(pred_man, 0:9), true = factor(y_test, 0:9))
print(conf_euc)
```

```
         true
predicted   0   1   2   3   4   5   6   7   8   9
        0 118   0   7   1   0   4   2   0   2   0
        1   0  66   0   0   2   0   0   0   0   1
        2   0   0  39   1   0   0   2   0   0   1
        3   0   0   0  23   0   4   0   0   2   0
        4   0   0   1   0  27   1   1   1   0   1
        5   0   0   0   1   0  13   0   0   0   0
        6   0   2   0   0   0   1  41   0   2   0
        7   0   0   0   0   1   1   0  34   0   4
        8   0   0   3   2   0   2   0   0  37   0
        9   1   0   0   1   4   0   0   1   1  41
```

```
print(conf_man)
```

```
         true
predicted   0   1   2   3   4   5   6   7   8   9
        0 118   0   6   1   0   5   3   0   4   0
        1   0  67   2   0   2   0   1   0   0   1
        2   0   0  35   0   0   0   2   0   0   0
        3   0   0   1  24   0   4   0   0   2   0
        4   0   0   1   1  27   1   0   1   0   1
        5   0   0   0   1   0  11   0   0   0   0
        6   0   1   0   0   0   2  40   0   2   0
        7   0   0   2   1   1   0   0  34   0   5
        8   0   0   2   0   0   3   0   0  34   0
        9   1   0   1   1   4   0   0   1   2  41
```

Test error is 0.122 (Euclidean) and 0.138 (Manhattan). The most frequent confusions under both distances are true 2 predicted as 0 (7 and 6 times), true 5 predicted as 0 or 3, true 4 predicted as 9, and true 9 predicted as 7.

Part c

```
show_digit <- function(x, main) {
  img <- matrix(x, 16, 16, byrow = TRUE)
  image(t(img[16:1, ]), col = gray(seq(1, 0, length.out = 256)),
        axes = FALSE, main = main, asp = 1)
}

wrong_euc <- which(pred_euc != y_test)
par(mfrow = c(1, 3), mar = c(0.5, 0.5, 2, 0.5))
for (i in wrong_euc[1:3]) {
  show_digit(X_test[i, ], sprintf("True %d, predicted %d", y_test[i], pred_euc[i]))
}
```

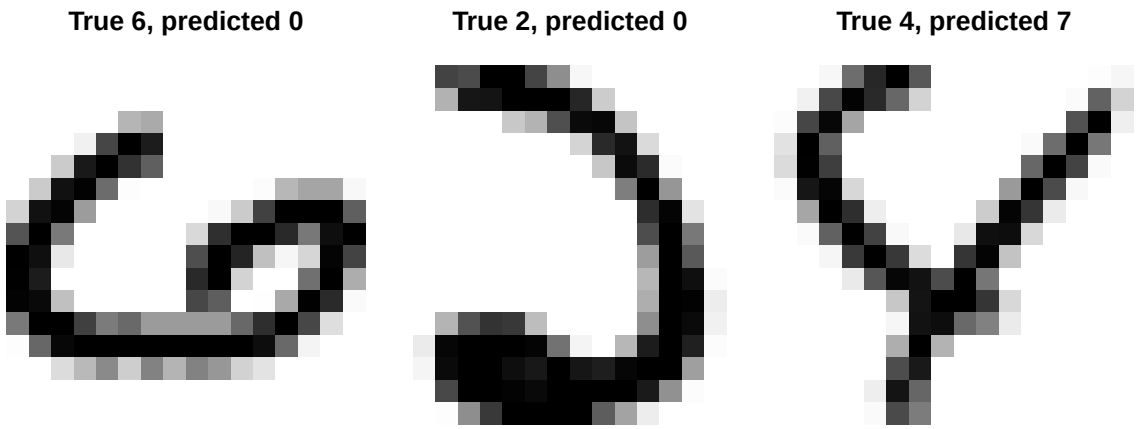


Figure 3: Three test images misclassified by Euclidean 5NN.

```
print(table(euclidean_wrong = pred_euc != y_test, manhattan_wrong = pred_man != y_test))
```

```
               manhattan_wrong
euclidean_wrong FALSE TRUE
          FALSE   428   11
          TRUE      3   58
```

```
both_wrong <- pred_euc != y_test & pred_man != y_test
print(sum(both_wrong & pred_euc != pred_man))
```

```
[1] 8
```

The mistakes are visually understandable: the 6 is tilted with a large closed loop that resembles a 0, the 2 is cramped with a curved stroke and heavy base that nearly closes into a loop, and the 4 has an open top shaped like a 7 or Y.

The two distances have similar total error (61 vs. 69 mistakes) but not identical mistakes: 58 images are misclassified by both, 3 only by Euclidean, and 11 only by Manhattan, and 8 of the 58 receive different wrong labels. So yes, similar total error can hide different errors on individual images, because Manhattan distance weights many small pixel differences more heavily relative to a few large ones and so changes which training images are nearest.

Question 5 Answers

Part a

The skill is saved as `stat432-plot-format/SKILL.md` (full content in Part c). Its description restricts it to STAT 432 homework plots and requires an explicit request by name; its rules cover margins, titles, axes, labels, colors, and sizes for R and Python without referring to specific questions.

Part b

```r
par(mar = c(4, 4.5, 0.5, 0.5), las = 1)
plot(
  ks, mse, type = "b", pch = 16, lwd = 2, col = "black",
  ylim = range(0, bias2, variance, mse),
  xlab = "Number of neighbors (k)", ylab = "Error at x0"
)
lines(ks, bias2, type = "b", pch = 17, lwd = 2, lty = 2, col = "#1F5A94")
lines(ks, variance, type = "b", pch = 15, lwd = 2, lty = 3, col = "#C84A16")
legend(
  "topright", bty = "n", lwd = 2, lty = c(1, 2, 3), pch = c(16, 17, 15),
  col = c("black", "#1F5A94", "#C84A16"),
  legend = c("MSE", "Squared bias", "Variance")
)
```

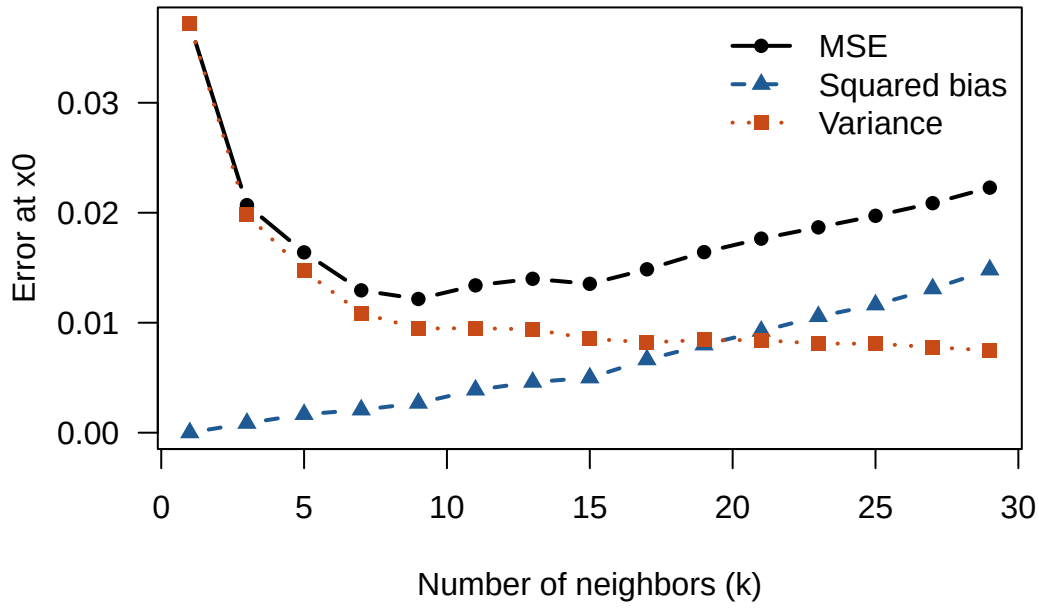


Figure 4: Question 1 plot revised with the stat432-plot-format skill.

Compared with the original Question 1 figure, the revision uses descriptive axis labels, horizontal tick labels, tighter margins, and different line types and point shapes so the three curves are distinguishable in grayscale; the data and plot type are unchanged.

The changes improve readability without altering content; the main gain is that the curves no longer depend on color alone.

Part c

```
---
name: stat432-plot-format
description: Improve the formatting of plots in STAT 432 homework reports produced in R or Python. Use
---

# STAT 432 Homework Plot Formatting

Change only how a plot looks. Keep the same data, quantities, and plot type.

## Rules

- Margins: Trim unused outer space so the panel fills the figure. Leave
  enough room that no axis label, tick label, or legend is clipped.
- Titles: Use a short title only if the report has no figure caption;
  otherwise rely on the caption and omit the title.
- Axes: Start axes at sensible limits covering all plotted values. Use a log
  scale only when values span several orders of magnitude. Keep tick labels
  horizontal and readable.
- Labels: Label every axis with the quantity in words or standard notation
  (e.g. "Number of neighbors (k)"), including units when they exist. Use a
  legend whenever more than one series is shown, placed where it does not
  cover data, without a box.
- Colors: Use at most four distinct, colorblind-safe colors (e.g. #1F5A94,
  #C84A16, #2E8B57, #7A4FA0). Also distinguish series by line type or point
  shape so the plot reads in grayscale.
- Sizes: Size figures for a PDF page (about 5-6.5 in wide, 3-4 in tall). Use
  line width about 2, points about 1-1.2, and text no smaller than 9 pt. Keep
  font sizes consistent across plots.
- Code: In R, set par(mar = ...), las = 1, and chunk fig-width/fig-height. In
  Python, use fig, ax = plt.subplots(figsize=...), ax.set_xlabel/ax.set_ylabel,
  and fig.tight_layout().
```

The rules target the problems that most often make homework plots hard to read in a PDF: wasted margins, vague axis labels, and series distinguished only by color. The description keeps the skill from triggering on unrelated plots, and the "change only how it looks" line prevents it from altering the analysis.